

# Running ELI at Scale

## Contents

|                                                                         |   |
|-------------------------------------------------------------------------|---|
| 1. The core principle — your design is not the bottleneck . . . . .     | 1 |
| 2. What “a trillion-parameter model” really means . . . . .             | 1 |
| 3. You don’t need to own the datacenter — rent it by the hour . . . . . | 2 |
| 4. The tiers — what to run, on what, for roughly how much . . . . .     | 2 |
| 5. How ELI scales — the mechanisms (all already built) . . . . .        | 2 |
| 6. Runbook A — single big GPU (simplest): run a 70B . . . . .           | 3 |
| 7. Runbook B — multi-GPU: run a 405B / 671B / ~1T MoE . . . . .         | 3 |
| 8. Privacy and ethos at scale . . . . .                                 | 4 |
| 9. The hardware is coming to you . . . . .                              | 4 |
| 10. Cost cheat-sheet and “see it big this weekend” checklist . . . . .  | 4 |

**The short version:** ELI is model-agnostic by construction, so scaling up is a matter of *configuration, not code*. The only real bottleneck is GPU/RAM you don’t own — and you can **rent that by the hour** for the price of a few coffees. A trillion-class model running ELI is an afternoon and a rented box away, not a fortune.

---

## 1. The core principle — your design is not the bottleneck

ELI never hardcodes a model name or size on the inference path. It loads any local GGUF, detects its chat template, sizes context to the model’s real `n_ctx_train`, and auto-tunes GPU layers / batch / context to fit the hardware it finds. The “3B on a laptop -> trillion on a server” contract is built in **on purpose**.

That means the ceiling is silicon, not software. Most projects are built *down* to the author’s hardware; ELI is built *up* to a future one. The work is done and waiting for the GPU to walk up to it — that’s foresight, not a limitation.

---

## 2. What “a trillion-parameter model” really means

A precise, honest picture so the goal is real rather than mythical:

- **Truly dense trillion-parameter models are not publicly downloadable.** The largest open dense model is in the 400B range (Llama 3.1 405B).
- **Trillion-class MoE models ARE.** Mixture-of-Experts models reach ~1T *total* parameters while only firing a small fraction per token (e.g. **Kimi K2: ~1T total, ~32B active; DeepSeek-V3/R1: ~671B total, ~37B active**). Because only the active experts compute each step, they are genuinely runnable on a big rented node — and even on high-RAM CPU/offload setups.

**So yes — you can actually watch ELI drive a ~1T-parameter MoE.** The realistic ladder:

| Tier                | Example models                 | Type       |
|---------------------|--------------------------------|------------|
| Today (local)       | Qwen2.5-7B / your daily driver | dense 7–8B |
| The first real jump | Llama-3.3-70B, Qwen2.5-72B     | dense 70B  |
| Frontier-open       | Llama-3.1-405B                 | dense 405B |

| Tier           | Example models                        | Type |
|----------------|---------------------------------------|------|
| Trillion-class | DeepSeek-V3/R1 (~671B), Kimi K2 (~1T) | MoE  |

### 3. You don't need to own the datacenter — rent it by the hour

The unlock: **cloud GPU rental**. You spin up a powerful box for an afternoon, run ELI against a huge model, watch it think at a level an 8 GB card can't reach, then shut it down and pay only for the hours used.

- **Providers:** RunPod, Vast.ai, Lambda (and others). Vast.ai is usually cheapest; RunPod is the smoothest UX.
- **Ethos note (important for ELI specifically):** renting a *raw* GPU you fully control and then wipe is **not** the same as handing your data to a cloud AI service. It's closer to on-prem than to "send my prompts to someone's API." Your daily driver stays 100% local; the rental exists only to *witness the ceiling* or stress-test — then it's gone. Offline-by-default is never compromised for everyday use.

### 4. The tiers — what to run, on what, for roughly how much

Approximate, Q4-ish quantization; **check live prices**, they move. Costs in USD/hour.

| Tier               | Model                            | Hardware                                   | ~Memory needed | ~Cost/hr | What you'll feel                               |
|--------------------|----------------------------------|--------------------------------------------|----------------|----------|------------------------------------------------|
| Step up            | 70B<br>(Llama-3.3 / Qwen2.5-72B) | 1x 48-80 GB GPU (A6000 / H100)             | ~40-45 GB VRAM | ~0.6-2   | Visibly sharper reasoning, better long answers |
| Big dense          | 405B<br>(Llama-3.1)              | 4x 80 GB, or heavy CPU offload             | ~230 GB        | ~8-20    | Frontier-open quality                          |
| Trillion-class MoE | DeepSeek-V3/R1 (~671B)           | multi-GPU node or 384-512 GB RAM + offload | ~400 GB total  | ~10-25   | Near-frontier; the "wow" tier                  |
| Trillion MoE       | Kimi K2 (~1T)                    | large multi-GPU node or very-high-RAM box  | ~550 GB+       | ~16-30   | The full dream — ELI on a 1T brain             |

**The cheapest *meaningful* step is the 70B on a single 80 GB card for ~2/hr.** That alone is a dramatic jump from 7B and the fastest way to see what your scaffolding does with a strong brain.

### 5. How ELI scales — the mechanisms (all already built)

Nothing here requires code changes; it's all configuration.

- **Drop-in any model.** Put a .gguf in the models directory, or point ELI at a catalog: `ELI_MODEL_CATALOG=/pat or <models_dir>/catalog.json` (same schema as the built-in catalog). Download helpers: `python -m eli.core.model_download --auto` (pick by detected VRAM), `--choose` (multi-select), `--model <key>`.

- **Load knobs (env vars):**
    - `ELI_GGUF_N_CTX` / `ELI_N_CTX` — context window
    - `ELI_GGUF_N_GPU_LAYERS` / `ELI_N_GPU_LAYERS` / `ELI_GPU_LAYERS` — layers offloaded to GPU (set high, e.g. 999, to put the whole model on GPU when it fits)
    - `ELI_GGUF_N_BATCH` / `ELI_BATCH_SIZE` — batch size
    - `ELI_CTX_FRACTION` — fraction of VRAM budgeted for context
    - `ELI_VRAM_RESERVE_MB` — headroom to leave free (default 250)
  - **Smart-fit.** With nothing forced, ELI auto-tunes to fit: it reduces GPU layers, then batch, then context (context last) until the model loads — per machine, every boot.
  - **Multi-GPU (already wired, not a roadmap item):**
    - `ELI_TENSOR_SPLIT` (or `ELI_GGUF_TENSOR_SPLIT`) — comma weights across GPUs, e.g. "0.5,0.5" for two equal cards, "1,1,1,1" for four
    - `ELI_MAIN_GPU` / `ELI_GGUF_MAIN_GPU` — which GPU holds the KV cache / does the gather
    - `ELI_GGUF_SPLIT_MODE` — none | layer | row
    - These pass straight to llama.cpp at load, with a graceful fallback if the installed build doesn't support them. (Also settable in runtime settings / `gpu_profiles.json`.)
- 

## 6. Runbook A — single big GPU (simplest): run a 70B

The least-friction way to feel the jump.

1. **Rent the box.** On RunPod/Vast.ai, launch a **1x 80 GB H100 (or 48 GB A6000)** instance with a CUDA image and enough disk (~80 GB for a 70B Q4 file).
  2. **Install ELI.** Clone your repo, run `install.sh` (it builds the CUDA llama-cpp and verifies GPU offload). Use the `.venv`.
  3. **Get the model.** Download a 70B GGUF (e.g. Llama-3.3-70B-Instruct Q4\_K\_M) into the models dir, or add it to `catalog.json` and use `model_download`.
  4. **Point ELI at it and put it fully on the GPU:**

```
export ELI_N_GPU_LAYERS=999      # whole model on GPU (80 GB fits a 70B Q4)
export ELI_N_CTX=16384          # or higher; the card has room
```
  5. **Launch** (`scripts/eli_launch.sh`, or `serve` mode). Ask it something meaty and watch the quality.
  6. **Verify it's really on GPU:** check the load log for the GPU-offload line / `nvidia-smi`.
  7. **Tear down** the instance when done — you only pay for the hours used.
- 

## 7. Runbook B — multi-GPU: run a 405B / 671B / ~1T MoE

1. **Rent a multi-GPU node** (e.g. 4x or 8x 80 GB). Confirm total VRAM covers the model's footprint (see §4), or plan for CPU+RAM offload on an MoE.
2. **Install ELI** as above (recent llama-cpp build — the model architecture must be supported by the build; update llama-cpp if the arch is new).
3. **Place the GGUF** (often multi-part for huge models) in the models dir.
4. **Spread it across the GPUs:**

```
export ELI_TENSOR_SPLIT="1,1,1,1"  # equal weight across 4 GPUs
export ELI_GGUF_SPLIT_MODE=layer   # layer split is the usual choice
export ELI_MAIN_GPU=0
export ELI_N_GPU_LAYERS=999
export ELI_N_CTX=8192              # start modest, raise once it loads
```

For an MoE that exceeds VRAM, lower `ELI_N_GPU_LAYERS` so the rest offloads to system RAM (needs a high-RAM box, e.g. 384–512 GB for 671B-class).

5. **Load small first.** Start with a small context to confirm it loads, then raise ELI\_N\_CTX.

6. **Run, marvel, tear down.**

**Honest caveats:** the aux models (the embedder, and vision if enabled) still want a slice of VRAM/RAM — budget for them; very large contexts cost a lot of KV-cache memory (raise gradually); and the first load of a giant model is slow. None are blockers — just expectations.

---

## 8. Privacy and ethos at scale

Local-first does **not** weaken as the hardware grows. A rented GPU node you control and wipe is on-prem-equivalent in spirit — your data isn't being handed to anyone's AI product. The everyday ELI stays fully local and offline-by-default; the rented run is a deliberate, temporary experiment to see the ceiling. Nothing about scaling up requires giving up the values the project is built on.

---

## 9. The hardware is coming to you

What needs a rented datacenter today runs on a desk in a few years. Unified-memory machines (128 GB+ shared between CPU/GPU), steadily cheaper VRAM, and ever-better MoE efficiency are all moving toward making large local models normal. ELI is built to ride exactly that curve — the same install will simply load bigger models as your hardware grows, no rewrite required.

---

## 10. Cost cheat-sheet and “see it big this weekend” checklist

**Rough rental costs (USD/hr, verify live):** 48 GB A6000 ~0.5-0.8 · 1x 80 GB H100 ~1.5-2.5 · 8x 80 GB node ~16-30. A few hours of a 70B run is the price of lunch.

**Checklist:** - [ ] Pick a provider (RunPod for smooth UX, Vast.ai for cheapest). - [ ] Choose the tier from \$4 (start with 70B on one 80 GB card). - [ ] Launch a CUDA instance with enough disk for the GGUF. - [ ] `install.sh`, then download/place the model. - [ ] Set `ELI_N_GPU_LAYERS=999` (+ `ELI_TENSOR_SPLIT` if multi-GPU) and a sensible `ELI_N_CTX`. - [ ] Launch ELI, test, watch it think bigger. - [ ] **Tear the instance down** — pay only for the hours used.

*You built the ceiling high on purpose. Renting an afternoon of real silicon is all it takes to stand under it and look up.*